

Basic of Computer Networks 2

Paul Blondel

CNAM

March 1st, 2019

'I think, in general, it's clear that most bad things come from misunderstanding, and communication is generally the way to resolve misunderstandings — and the Web's a form of communications — so it generally should be good.'

Tim-Bernes Lee - Creator of the WEB

1 Data transmission

- Exchange organization
- Types of connection
- Parallel transmission or serial transmission
- Synchronous and asynchronous transmissions

2 Notions about protocols

- Delimitation of the data
- Control of integrity

3 Exchange control

- Send and wait approach
- Anticipation approach
- Flux control
- Control of the link (or signalization)

1 Data transmission

- Exchange organization
- Types of connection
- Parallel transmission or serial transmission
- Synchronous and asynchronous transmissions

2 Notions about protocols

- Delimitation of the data
- Control of integrity

3 Exchange control

- Send and wait approach
- Anticipation approach
- Flux control
- Control of the link (or signalization)

Transmitting information between two computers can be done in multiple ways:

- simplex: each computer has one role: emitting data (source) or receiving data (receptor)



- half-duplex: computers can alternate their role: once source, once receptor



- full-duplex: computers can send and receive data in the same time



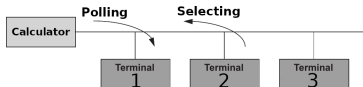
Point-to-point connection:

Each computer is linked to only one another.



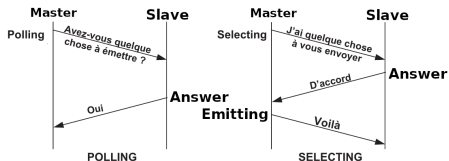
Multiple connections:

Several computers share the same link.



Notes

May have conflicts: it is necessary to control the access! With polling/selecting for instance.

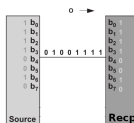


Peer-to-peer communication:

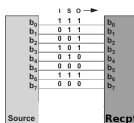


All machines can send towards others calculators: high risk of collisions!
Here access policy is decentralized (ex: local networks).

In case the transmission is serial: the link only permit to transmit data bit after bit (like a bus). The bit word is send bit after bit ...



With parallel transmission: all the bits of the bit word can be send in once.



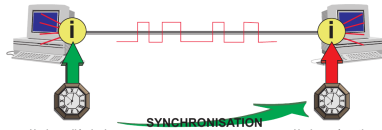
8 steps to transfer "o" (serial). 1 step to transfer "o" (parallel).

Notes

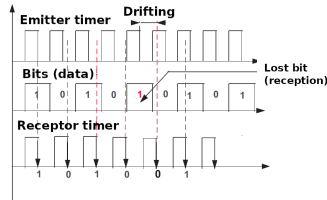
- Parallel: Faster, but can suffer from delay skew if computers are far one another!
- Serial: Cheaper and more sure. But it requires an electronic interface to "serialize" and "deserialize" the bits.

All the bits are sent through the link/connection with a certain rhythm (or timing). This rhythm is given by a timer.

Both the source and the receptor have timers and they need to be synchronized!



If they are not synchronized: the message might not be correctly received!



There are two types of transmissions:

- Synchronous transmission (timers are synchronized)
- Asynchronous transmission (timers are autonomous)

Synchronous transmission:

- A synchronous signal is sent during of the transmission using a specific line.
- Or the synchronous signal is deduced from the message itself (more complex to realize)

Note

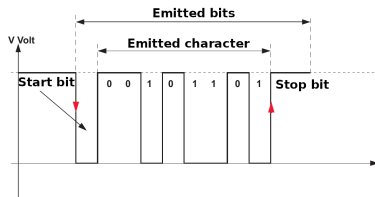
In both approaches the first bits used to help the synchronization

Asynchronous transmission:

Here the timers of the source and the receptor are independent each other (the beginning of the message synchronized the two timers, then timers are let free to drift).

The principle of this technique is: there is an interval of time in which the drift (caused by dephasing) does not modify the message.

This interval of time is big enough to transmit a character (one piece of information).



With asynchronous transmission: each character is preceded and followed by delimiters: start and stop bits.

Asynchronous transmission: easier approach than synchronous transmission (requiring an explicit synchronization).



Left: Asynchronous transmission (characters are sent and displayed while the keys are pressed)

Right: Synchronous transmission (all the letters are first buffered then they are sent (word completed) and displayed).

Asynchronous transmission: easier but more bits are used for control!

Synchronous transmission: optimized but more difficult to make it work.
And asynchronous and synchronous transmission have different usages as well!

$$Eff = \frac{Dr}{Dn}$$

Dr: number of useful bits that can be transferred

Dn: total number of bits (control bits included)

$$Eff_{Asynchronous\ trans} < Eff_{Synchronous\ trans}$$

(For long data transmissions)

1 Data transmission

- Exchange organization
- Types of connection
- Parallel transmission or serial transmission
- Synchronous and asynchronous transmissions

2 Notions about protocols

- Delimitation of the data
- Control of integrity

3 Exchange control

- Send and wait approach
- Anticipation approach
- Flux control
- Control of the link (or signalization)

We saw how to transmit data between two computers.



We need a protocol to ensure a reliable exchange of data between the two systems.

During the data exchange the protocol ensures:

- The delimitation of the blocks of exchanged data
- The control of the integrity of the received data
- The organization and the control of the exchange
- Eventually, the control of the link

Like in asynchronous transmission where the data is between a "start" and "stop" bit: in synchronous transmission a "flag" delimit the data (to identify the begin and the end of the block).



The flag is useful for two things:

- Delimit data
- When sent with not data: flags help synchronize the timer of the receptor

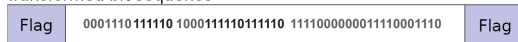
The flag is a specific series of bits. This series of bits must not be present in the message itself! Otherwise the data will be corrupted.

To avoid that: there is the transparency mechanism. This mechanism transforms any flag-like series of bits in the data.

Initial bit sequence



Transformed bit sequence



Here one "0" bit is added every 5 "1" bits. (because 6 "1" bits is the flag).

It's important to ensure the received data has not been altered (by electromagnetic disturbance, loss of synchronization, etc.).

There are some techniques to control the integrity of the data.

But first let's talk about the Binary Error Rate (BER):

$$BER = \frac{\text{number of erroneous bits}}{\text{number of transmitted bits}}$$

ex: bit word sent: 011010110101 bit word received: 001010010101

$$BER = \frac{2}{12} = 0.18$$

But in reality the *BER* varies from 10^{-4} (phone network) to 10^{-9} (local computer network).

For one bit BER = probability of having erroneous bits $(1 - BER)$ = probability of having correct bits

So if $BER = 10^{-4}$ and we have 800 bits to transmit.

Probability of receiving a correct block is:

$$(1 - BER)^{800} = (1 - 10^{-4})^{800} = 0.923$$

Probability of receiving a bad block is: $1 - 0.923 = 0.077$

So how to detect an error during the transmission of a block?

Four techniques to detect transmission errors:

- ① Detection using an echo
- ② Detection using repetition
- ③ Detection using a computed key
- ④ Detection and autocorrection using a code system

Detection using a echo:

The receptor resend the received message. The source checks if the resent message is the same as the originally sent message. If not: message is send again.

Detection using repetition (statistical approach):

The source send several times the message. If all the messages are the same: no errors occurred! If not: a bunch of messages is sent again.

Detection using a computed key:

A supplementary information is send within the message: a computed key. The value of this key depends on the message itself.

The receptor can check if the transmitted key match the message. If it does not: the message is sent back.

Two main techniques to do that:

- 1 Parity bit technique (or VRC)
- 2 Cyclic redundancy check technique (or CRC)

Parity bit technique:

The principle is the following: a bit is added at the end of the sequence of bits of the message.

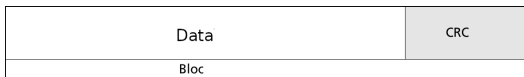
key = sum of the bits equal to 1 modulo 2

Ex: 1001111 In this case we have 5 bits equal to 1. So $key = 5\%2 = 1$

Note

Can only detect errors on impair number of bits messages. Used mainly in asynchronous transmission. For synchronous transmission a variant exists: the LRC (Longitudinal redundancy check technique)

Cyclic Redundancy Technique (CRC):



The CRC is deduced from a math operation applied to the data itself.

The block of data N bits is considered as a polynomial of degree $N-1$: $P(X)$.

This polynomial is divided by a polynomial generator $G(X)$ (according to boolean arithmetic). The rest of the division $R(X)$ is the CRC. The CRC is then transmitted with the block.

The transmitted CRC is compared to the received CRC: if not equal there is an error!

We perform $\frac{(P(X)+R(X))}{G(X)}$ if it is equal to 0, then the message is OK!

Note

In boolean arithmetic, addition and subtraction are equivalent

$$P(X) - R(X) \equiv P(X) + R(X)$$

Example: we want to protect the message 110111 using a polynomial

$$G(x) = x^2 + x + 1$$

$$110111 \Rightarrow P(x) = x^5 + x^4 + 0 \times x^3 + x^2 + x^1 + x^0$$

In order to allow the addition of the message and the key we multiply $P(x)$ by x^m (with m = degree of the generator polynomial) so we have:

$$P(x) \times x^2 = x^7 + x^6 + 0 \times x^5 + x^4 + x^3 + x^2.$$

$$\begin{array}{r}
 x^7 + x^6 + 0 + x^4 + x^3 + x^2 + 0 + 0 \mid x^2 + x + 1 \\
 \begin{array}{ccccccc}
 x^7 & x^6 & x^5 & & & & \\
 & & \downarrow & \downarrow & \downarrow & & \\
 & x^5 & x^4 & x^3 & & & \\
 & & \downarrow & \downarrow & \downarrow & & \\
 & x^5 & x^4 & x^3 & & & \\
 & & & & x^2 & 0 & 0 \\
 & & & & x^2 & x & 1 \\
 & & & & & x & 1
 \end{array}
 \end{array}$$

$P(x) \times x^2$ divided by $G(X)$ give a rest $R(x) = X + 1$ (so the CRC=11).

So the block to transmit is now the following: 11011111

Note

The best generator polynomial is a tradeoff ensuring:

- a low probability of non-detection
- not penalizing too much the transmission

Detection and auto-correction using a code system:

Original binary words are substituted by new binary words with auto-correcting capacities.

Original	Autocorrect code
00	10011
01	10100
10	01001
11	01110

Let's imagine we transmit the binary word 10011. And let's imagine the receptor receives the binary word with one bit wrong. So it can be: 10011 10001 10111 11011 or 00011.

None of these codes match 10011 but for all these cases the closest code is 10011 (Hamming distance = 1 with this one!)

1 Data transmission

- Exchange organization
- Types of connection
- Parallel transmission or serial transmission
- Synchronous and asynchronous transmissions

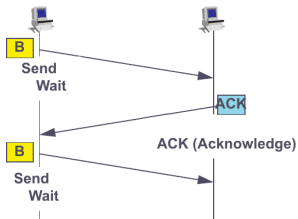
2 Notions about protocols

- Delimitation of the data
- Control of integrity

3 Exchange control

- Send and wait approach
- Anticipation approach
- Flux control
- Control of the link (or signalization)

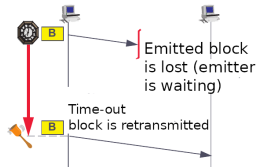
With the send and wait approach the source sends a block to the receptor.



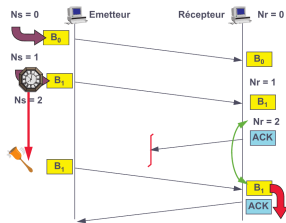
Once received the receptor sends back ACK (to tell it it can receive the next block). The source waits a certain amount of time this ACK.

If it does not receive it, it sends again the block (Re-transmission Time Out).

But if ACK is lost, the same block is sent again ...

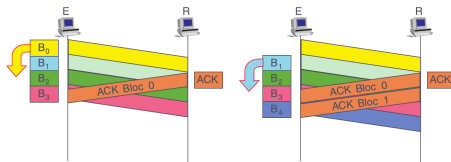


To avoid duplication, each block has a number N . If the block $N=1$ has already been received, a new block $N=1$ will not be added after ... In order to avoid re-transmission conflicts.

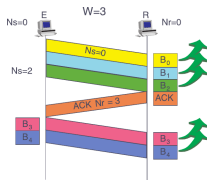


ACKs are also numerated in practice.

In this case all the blocks are sent one after the other, without waiting for the first ACK.



Blocks can also be acknowledged by groups, for further optimization. With this approach, the transfer is optimized.



W : window of anticipation, number of blocks the emitter can memorize waiting for acknowledgment (ACKs).

In case of transmission errors, two techniques:

- 1 the bad block is rejected and block is re-transmitted
- 2 the group of blocks is rejected and all the blocks are re-transmitted

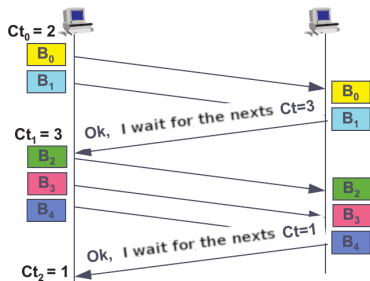
The first approach is the selection rejection. The transmission is optimized but it requires bigger memory (other blocks of the group need to be kept in memory). And blocks need to be re-ordered by the receptor (requires computation power).

With the second approach the memory is optimized and the computation power of the receptor is minimized (no need to reorder the blocks!).

During a transmission, the receptor has a certain number of free slots in its buffer (memory).

Because the receptor has also other tasks to realize, not all the memory may be available and some blocks may be lost!

That's why it's important to control the emitting cadence of the source.



Before transmitting any data it is necessary to make a link between the two computers. And once the transmission is finished, the link is closed.

To manage this link we send signalization information (transmitting through the same channel as the data). Sometimes the signalization information is sent through its own channel.

